

RNN (Recurrent neural network)
 RNN is a type of sequential model to work on sequential data.

consider the following tabular dataset

IQ	marks	Gender	Placement.
7.9	83	M	Yes.
7.9	94	F	Yes
8.3	62	M	No.
8.2	74	F	No.

In the above dataset if the sequence is changed like following there will be no impact on output.

Gender	Marks	IQ	Placement.

Hence in normal neural network, if the sequence is changed for normal input layer then there will be no impact on output.

But there are some certain dataset where it can not work such as text, speech. like following. →

eg. Hi, my name is Swapnil.
 → my hi is name swapnil.

After changing sequence

Due to this RNN is widely used in NLP.

Q. Why to use RNN instead of ANN?

① Suppose we are trying to solve the problem using ANN. and the dataset consists of data having words from 10 to 100 in training. But what if in testing a data of 200 text comes? Initially we had set 100 neurons in 1st input layer. we can't change the same to 200.

② In ANN, all the words are feeding to the neural network at same time. so that it is losing its sequence. so NN model will lose its sequential data meaning.

To overcome the above problems RNN is used.

Q. Applications of RNN: →

- ① sentiment analysis.
- ② Next word prediction in Google, Gmail, whatsapp.
- ③ Google language translator.

RNN Architecture: →

consider a sentiment dataset:

review	Sentiment.
movie was good.	1
Movie was bad.	0
Movie was not good.	0



so our vocabulary of 5 words corpus.

so we can represent the corpus like

movie	→	[1 0 0 0 0]
was	→	[0 1 0 0 0]
good	→	[0 0 1 0 0]
bad	→	[0 0 0 1 0]
not	→	[0 0 0 0 1]

so the review 1 becomes.
movie was good.

[1 0 0 0 0] [0 1 0 0 0] [0 0 1 0 0]

timestamps → $t=1$ $t=2$ $t=3$

1st review representation = (3, 5)
 ↗ ↖
 no. of no. of
 timestamps features.

For simplicity, we will give notation: →

	Review	Sentiment.
x_1	x_{11} Movie x_{12} was x_{13} good.	1
x_2	x_{21} Movie x_{22} was x_{23} bad.	0
x_3	x_{31} movie x_{32} was x_{33} not good. x_{34}	0

Unlike

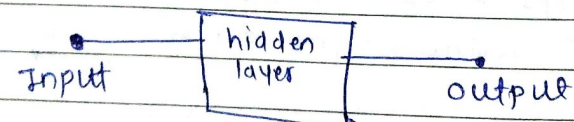
In ANN, we are feeding our all datapoints in single timestep. but in RNN, we are feeding single datapoint/word in single timestep.

It means,

for time $t=1$, we will feed x_{11} .
 for time $t=2$, we will feed x_{12} .
 for time $t=3$, we will feed x_{13} .

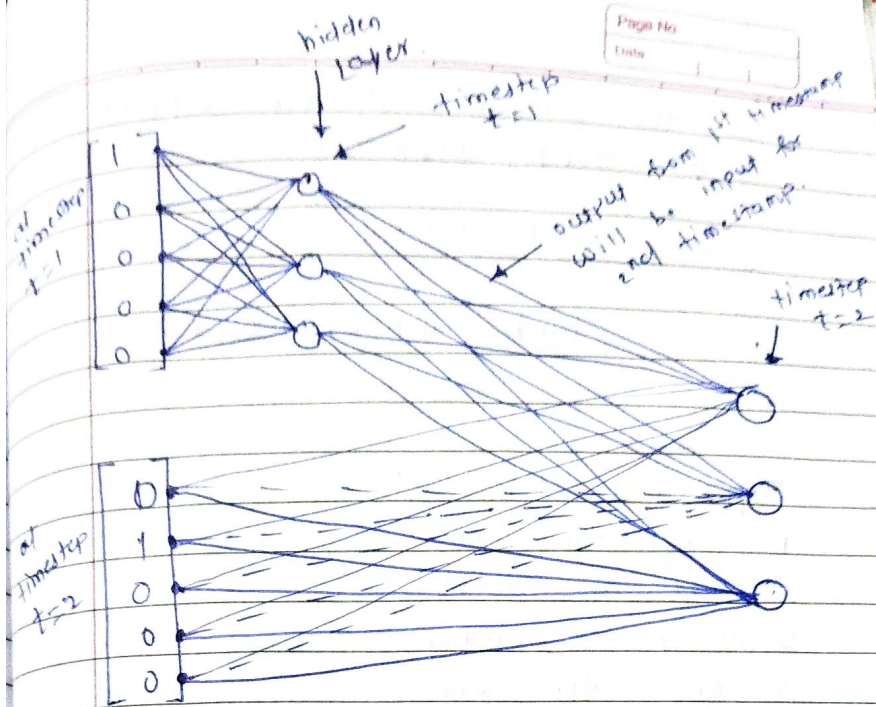
Now consider $t=1$. so we are feeding the vector [1 0 0 0 0].

roughly, the RNN has following structure.

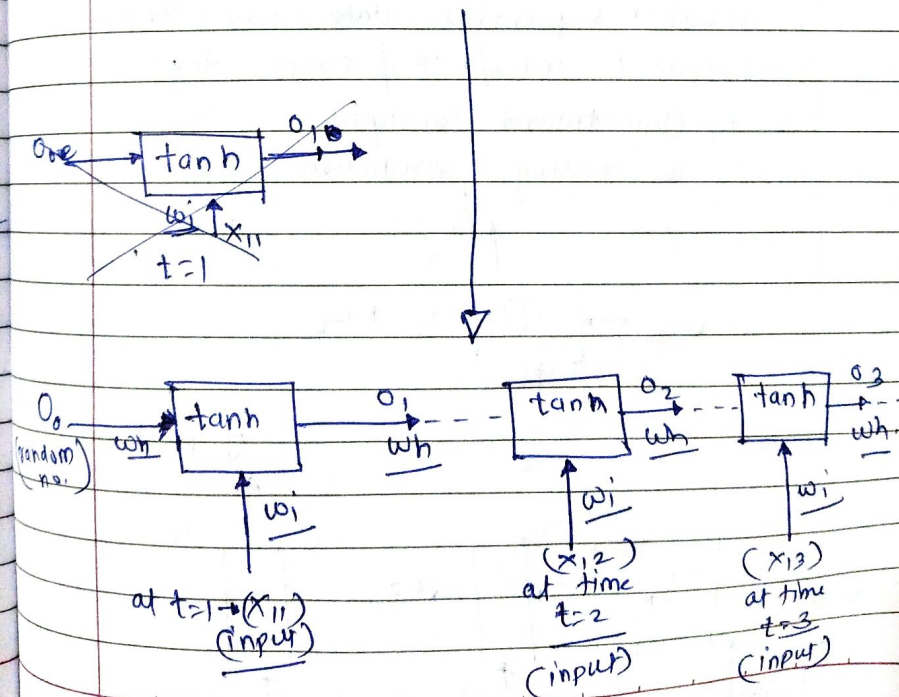


Consider the hidden layer has 3 neurons. having tanh activation function.

so at timestamp $t=1$, and for input vector [1 0 0 0 0] the structure will look like.



The simpler picture will be like follow



So the simpler eqⁿ for output becomes

$$o_1 = \tanh(w_h \times o_0 + w_i \times x_{11})$$

$$o_2 = \tanh(w_h \times o_1 + w_i \times x_{12})$$

$$o_3 = \tanh(w_h \times o_2 + w_i \times x_{13})$$

Types of RNN: $\rightarrow$

1. Many to one.
2. one to many
3. many to many
4. one to one

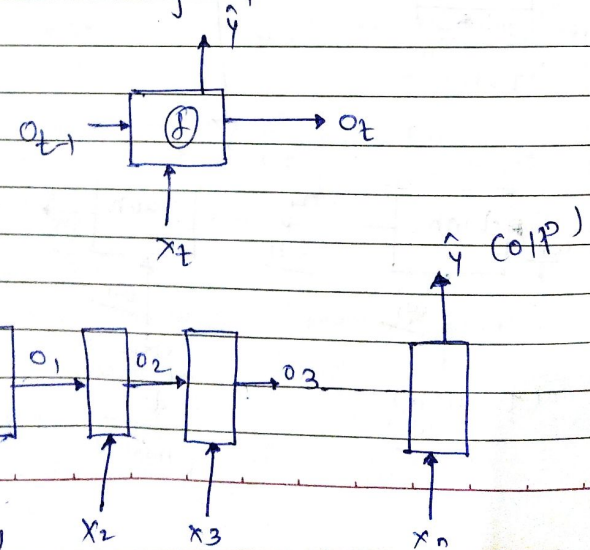
1. many to one: $\rightarrow$

input $\rightarrow$ sequential data like sentence.

output $\rightarrow$ int / num / single text

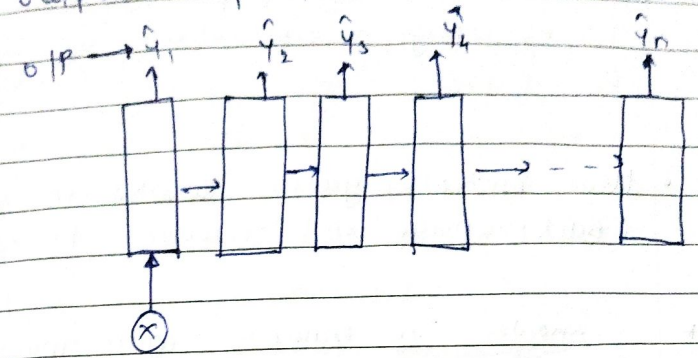
eg. ① sentiment analysis.

② Rating prediction.



② ④ One to Many

2. One to Many: →
Input → non sequential data. eg. Image.
Output → Sequential data like words.

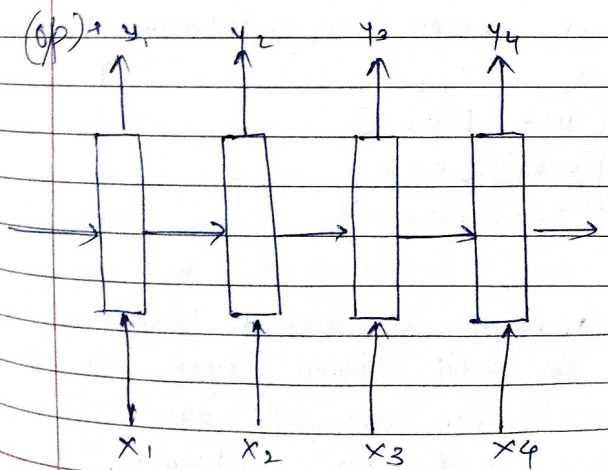


Use case: → Image captioning. (textual representation of image)

3. Many to Many

Input → ~~non~~ sequential data [paragraph]
Output → Sequential data [summary]

Use case: → Text summarisation, google translate



4. One to One

Input → non-sequential data
Output → non-sequential data.
eg. Image classification.

• How backpropagation works in RNN (Backpropagation through time).

→ considers an example with many to one RNN. eg. sentiment analysis.

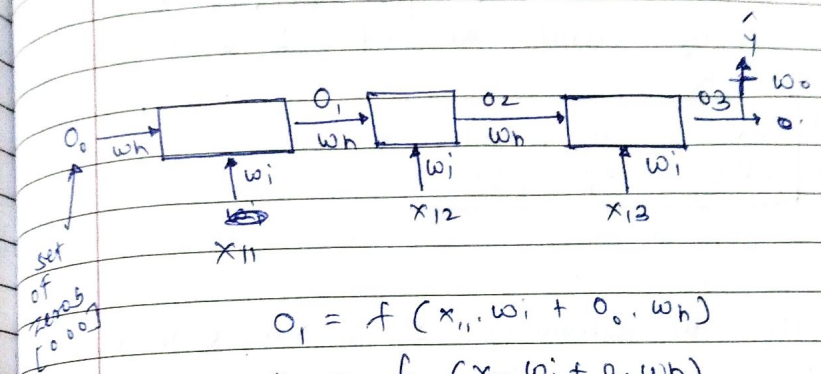
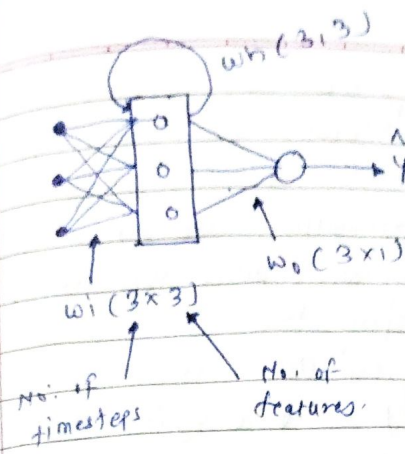
Text	Review
cat mat rat	1
rat rat mat	1
mat mat cat	0

vocabulary → cat mat rat
[1 0 0] [0 1 0] [0 0 1]

so, our vector representation becomes

	x_{11}	x_{12}	x_{13}	y
x_1	[1 0 0]	[0 1 0]	[0 0 1]	1
x_2	[0 0 1]	[0 0 1]	[0 1 0]	1
x_3	[0 1 0]	[0 1 0]	[1 0 0]	0

Now, consider a neural network having 3 RNN hidden layers in it and one node in o/p as it is a binary classification problem.



$$o_1 = f(x_{11} \cdot w_i + o_0 \cdot w_h)$$

$$o_2 = f(x_{12} \cdot w_i + o_1 \cdot w_h)$$

$$o_3 = f(x_{13} \cdot w_i + o_2 \cdot w_h)$$

$$\hat{y} = \sigma(o_3 \cdot w_o) \quad \text{--- (A)}$$

Loss = $-y_i \log \hat{y}_i - (1 - y_i) \log (1 - \hat{y}_i)$
function

Now, after calculating this loss, we have to minimise the loss using gradient descent.

$$w_i = w_i - \eta \frac{\partial L}{\partial w_i}$$

$$w_h = w_h - \eta \frac{\partial L}{\partial w_h}$$

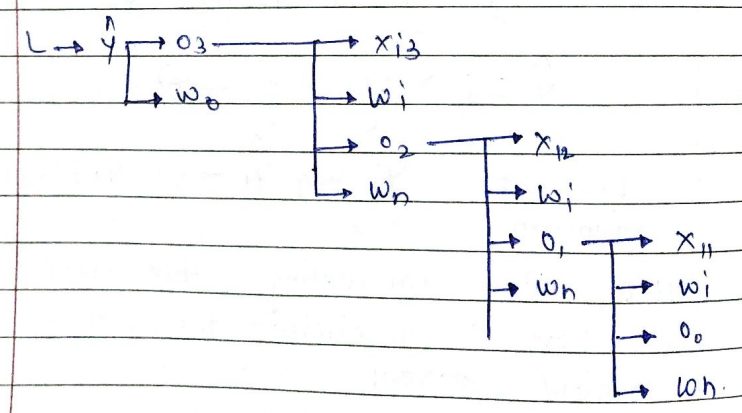
$$w_o = w_o - \eta \frac{\partial L}{\partial w_o}$$

In backward propagation, we have to calculate the w_o , w_i and w_h in order to minimise the loss.

Now, from eqⁿ (A), loss depends upon $\hat{y}$ and $\hat{y} = \sigma(o_3 \cdot w_o)$ so if we change w_o , $\hat{y}$ will be change and at end L will be changed.

$$\frac{\partial L}{\partial w_o} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_o}$$

Now for calculating $\frac{\partial L}{\partial w_o}$, we will have the following tree like structure.



$$\frac{\partial L}{\partial \hat{y}}$$

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_i} +$$

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_i} +$$

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_i}$$

similarly,

$$\frac{\partial L}{\partial w_h} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_h} +$$

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_h} +$$

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_h}$$

This is how BPTT take place in RNN.

Long Term Dependency Problem in RNN

Consider In backpropagation, we got the eqn like following.

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_i} +$$

(short term)

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_i} +$$

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_i}$$

Last term. (long term)

consider the above last term having partial derivative upto o_1 from o_3 . If we had a 100 data inputs i.e. from o_1 to o_{100} . then this last term would be.

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_{100}} \cdot \frac{\partial o_{100}}{\partial o_{99}} \cdot \frac{\partial o_{99}}{\partial o_{98}} \dots \frac{\partial o_1}{\partial w_i}$$

and we know that

$$o_t = \tanh [x_i w_i + o_{t-1} w_h]$$

which means we are calculating the tanh of the summation. We know that value of tanh ranges betⁿ [0 to 1] for o_{100} , we will have so many numbers which are near to zero.

$$= 0.9 \times 0.8 \times 0.6 \times 0.9 \times 0.4 \times 0.3 \dots \rightarrow 0.2$$

= very low number.

This problem is called vanishing gradient problem.

and as compared to short term, the long term has very low value. so in backpropagation, loss is dependent on short term, not on long term.

Therefore RNN can not remember more no. of terms/timesteps.

The above problem is overcome in the structure of LSTM.